PatchTST for Parkinson's Speech Classification

A Plain-English Technical Report

Arannamoy Mondal
arannamoymondal13@gmail.com

12 October 2026 • Report 1.0 • Experiment v2.0

Abstract

This report explains one model from start to finish: a compact PatchTST classifier that uses speech to distinguish people with Parkinson's disease from elderly healthy controls. It explains how recordings are checked, how repeat sessions are linked to the same person, how sound becomes a spectrogram, how the model is trained, and how many short-window scores become one decision per person.

The main experiment uses 773 recordings from 46 people: 25 patients and 21 controls. Five speaker-disjoint folds keep each test person's recordings out of the model that predicts that person. The pooled speaker-level results are F1 97.96%, recall 96.00%, specificity 100.00%, precision 100.00%, and accuracy 97.83%. There is one missed patient and no false alarms among controls.

The saved results reproduce. However, the recordings also contain strong group-related acoustic differences, and recording dates are separated by class. The report therefore explains both the leakage controls that were checked and the recording-condition problem that remains. These are internal results on this corpus, not an independent clinical validation.

1 One question, one model, one input

The question is simple: *using speech recordings, can this model separate people labeled with Parkinson's disease from elderly healthy controls in this dataset?* The recorded diagnosis supplies the training label. The model predicts that label; it does not create a new medical diagnosis.

Model	Corrected, channel-independent PatchTST; experiment v2.0.
Input	A 64-band log-mel spectrogram, cut into 400-frame windows.
Output	One model score per window, combined into one speaker score.
Decision	Score at least 0.5: PD; score below 0.5: elderly control.
Evaluation unit	One person, not one recording or one short window.

How to read this report. Sections 2–7 describe the data and model. Sections 8–11 explain training, OOF predictions and score averaging. Sections 12–15 give the results and their limits. Sections 16–17 explain reproduction and the main terms. Diagrams use black lines, labels and shapes so they remain readable in a black-and-white printout.

2 The whole pipeline in one picture

Read Figure 1 from top to bottom. A *manifest* is simply a table with one row for each recording. A *fold* is one group of people held aside for testing. The same model design is trained separately for each fold.

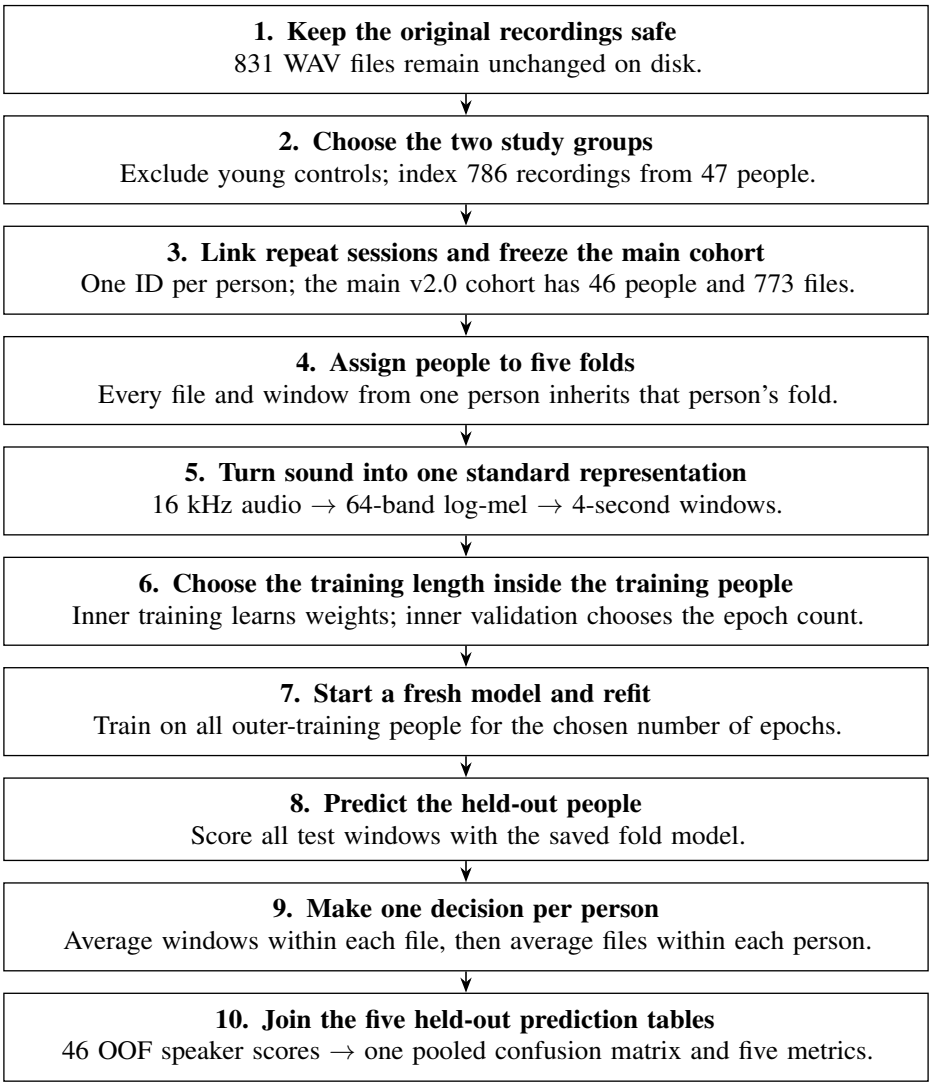


Figure 1: The complete single-model workflow. Steps 6–9 run once for each held-out fold. OOF means that a person's score comes from the model trained without that person.

Two kinds of work happen here. Fixed audio conversion can be cached once because it learns no statistics from the dataset. Learning weights and choosing how long to train happen inside each training split. Section 14 explains why that boundary matters.

3 Which recordings and people enter the study?

The source is the Italian Parkinson's Voice and Speech corpus [1], associated with the VoxTester and speech-intelligibility work [2, 3]. The folder name "28 People with Parkinson's disease" is a folder count, not the final number of distinct patients. Linking repeat sessions gives 25 patient IDs.

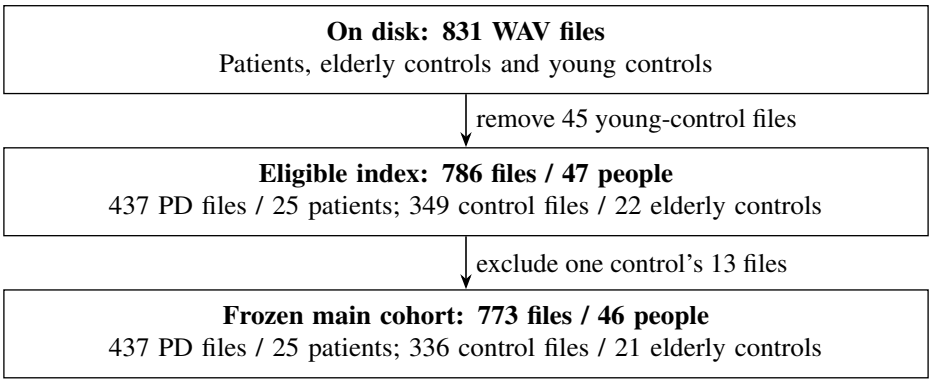


Figure 2: The counts at each stage. Excluded recordings remain on disk; exclusion means that their rows do not enter the experiment.

Why exclude young controls?

The 15 young controls are aged 19–29. Including them could reward a model for detecting age rather than disease. This study uses elderly controls as the comparison group. That reduces an obvious age difference, but does not guarantee that every age or recruitment effect is removed.

Why does the main result contain 46 rather than 47 people?

The completed v2.0 experiment froze a common, task-available cohort for the original project comparison. One elderly control, Vitantonio D, has no B1/B2 passage recordings, so that cohort excluded him. This report keeps that exact 46-person cohort to match the saved result. PatchTST itself does not require every person to have a passage; this exclusion is an evaluation-design choice.

Speech task	Meaning in everyday words	Files	Codes
Passages	Read a longer piece of text	94	B1, B2
Repeated syllables	Repeat short sounds such as /pa/ and /ta/	98	D1, D2
Vowels	Hold a vowel sound	485	VA1–VU2
Words	Read a word list	49	PR1
Phrases	Read short phrases	47	FB1
Total	All these tasks enter PatchTST	773	

Labels. PD is 1; elderly healthy control is 0. Labels come from the exact group folder. Searching for the word "Parkinson" anywhere in a path would be wrong because the dataset's root folder also contains that word.

4 One person must keep one identity

The same person can have several recordings, several tasks, or more than one recording session. Those are repeated observations of one person. They must not be mistaken for different people when training and testing are separated.

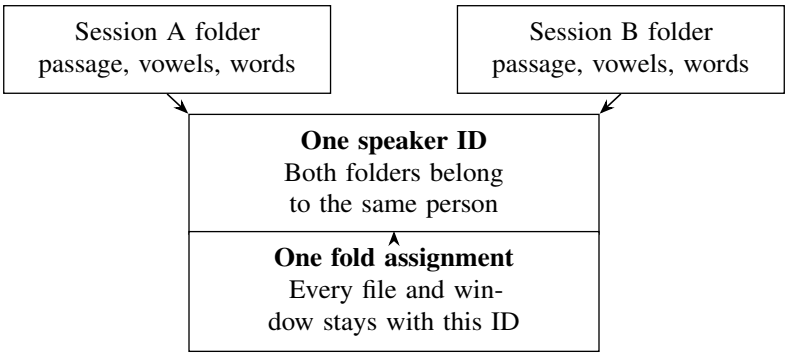


Figure 3: Repeat sessions are joined before folds are created. The person is the unit of separation, even when dates, folders or token spellings differ.

The identity checks used here

The index reads the task, person token and recording time from each filename. Known text problems are corrected in the index; the original file is not renamed. Folders are linked using the same cohort and matching person tokens, with the documented name/birth-year/sex matches used for known repeat sessions.

- Vito S with token ending `46M` is linked across his two sessions.
- Nicola S and Roberto R are linked despite two spellings of their tokens.
- The other Vito S, whose token ends in `47M`, stays a different person.
- Antonia G has one inconsistent export token. The folder's usual token is used, and that recording receives a correction flag.

These are the project's working identity rules. The audit checks them against the known corpus records; it does not claim biometric proof of every identity.

A useful file table does not become a model input

The manifest stores the speaker ID, label, task, original sample rate, duration, date, fold and cache path. Age, sex and spreadsheet fields help check records. **The network receives only the spectrogram values.** It does not receive a name, filename, age, sex, recording date, reading-speed score or native sample rate.

A `sheet_flag` records a review mark in the source spreadsheet. Four eligible controls have this flag; three are present in the main cohort. They remain in the main evaluation. Section 15 shows the check that removes them from scoring.

The rule to remember: a new folder does not mean a new person. Create the person ID first; then split the people.

5 Preprocessing: turning sound into a useful table

Audio is a long list of sample values. A spectrogram turns that list into a time-by-frequency description: it shows which frequency ranges carry energy at each short time step. The model uses the numerical table, not a screenshot.

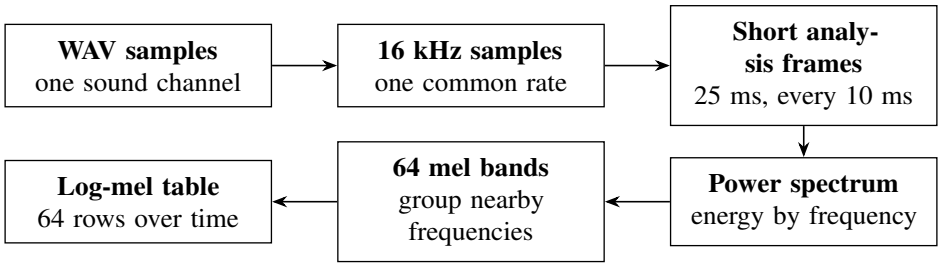


Figure 4: Fixed preprocessing applied separately to each recording. "ms" means milliseconds; 1,000 milliseconds make one second.

Read and resample

The corpus WAVs are mono, 16-bit PCM. They are read as 32-bit floating-point numbers on a common digital scale. If the original sample rate is 44,100 Hz, the waveform is resampled to 16,000 Hz using `soxr_hq`. Native 16,000-Hz files are read directly. Resampling changes the number of samples per second; it does not intentionally make the person speak faster or slower.

Measure frequency energy in short frames

A 400-sample analysis window covers 25 ms at 16 kHz. The next frame starts 160 samples, or 10 ms, later. A 512-point Fourier transform separates frequency content. Sixty-four overlapping mel filters combine that energy into bands spaced more closely at lower frequencies. The implementation uses librosa's default Hann analysis window and centered-frame convention.

Use the same logarithmic reference for every file

Power values are converted to decibels using a fixed reference of 1.0 and a power floor of 10^{-10}. The floor avoids taking a logarithm of zero. In simplified form, for a band power P,

$$X = 10\log_{10}\left(\frac{\max(P, 10^{-10})}{1.0}\right).$$

There is no per-file peak normalization and no per-file maximum as the decibel reference. Recording-relative decibel clipping is disabled. This keeps digital level differences visible. A quiet voice can be informative, but gain and microphone distance can change the same numbers. Digital level is not calibrated physical voice loudness.

Two different windows: the 25-ms analysis window builds each spectrogram frame. The 4-second model window later groups 400 frames into one model input. They do different jobs.

6 Windows and patches: three different time scales

The spectrogram has 64 frequency-band rows and a variable number of time columns. Roughly 100 columns describe one second. Centering can add a boundary frame, so full-file frame counts are only approximately duration times 100.

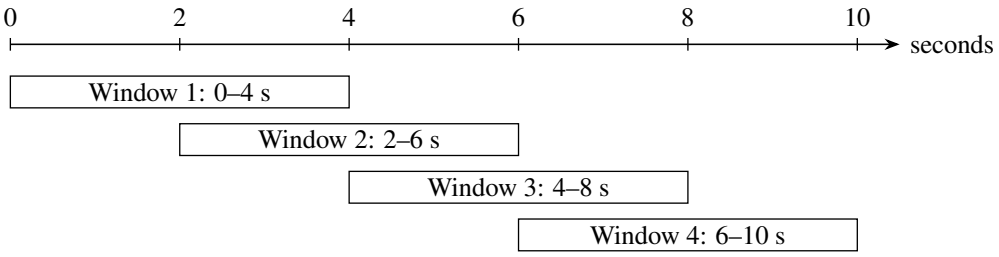


Figure 5: A simple timeline sketch. Four-second windows move forward by two seconds, so nearby windows share sound. The actual code works in frame indices and adds a final window when needed to cover the end.

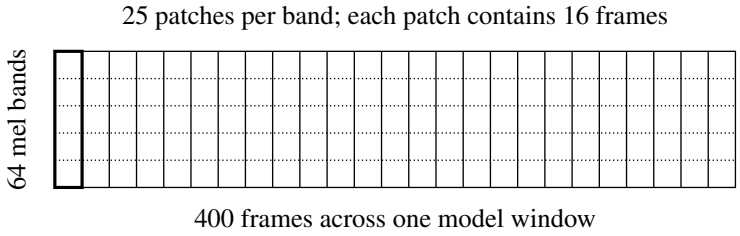


Figure 6: An input-shape sketch, not a measured spectrogram. Only a few horizontal guide lines are drawn. The real numerical input has 64 rows.

- **Model input:** 64 bands $\times$ 400 frames. That is 25,600 numerical values, not 25,600 separately designed acoustic features.
- **Short files:** if fewer than 400 frames are available, the final frame is repeated until the window is full. This is called edge padding.
- **Training:** keep at most eight evenly spaced windows from each file.
- **Validation and testing:** keep every window, including final-tail coverage.
- **Inside PatchTST:** split each band into 25 non-overlapping patches of 16 frames. Patch starts are 160 ms apart; analysis frames themselves overlap.

Why split people first? Overlapping windows can contain much of the same audio. A random window split could place near-identical sound in both training and testing. In this pipeline, every window follows its person's existing fold.

7 What the PatchTST model actually does

PatchTST's original idea is to read short time patches and reuse one encoder across channels [4]. Here it is adapted to classification: each mel band acts as one channel. This is a compact project implementation, not a claim to have invented PatchTST or reproduced its forecasting benchmark.

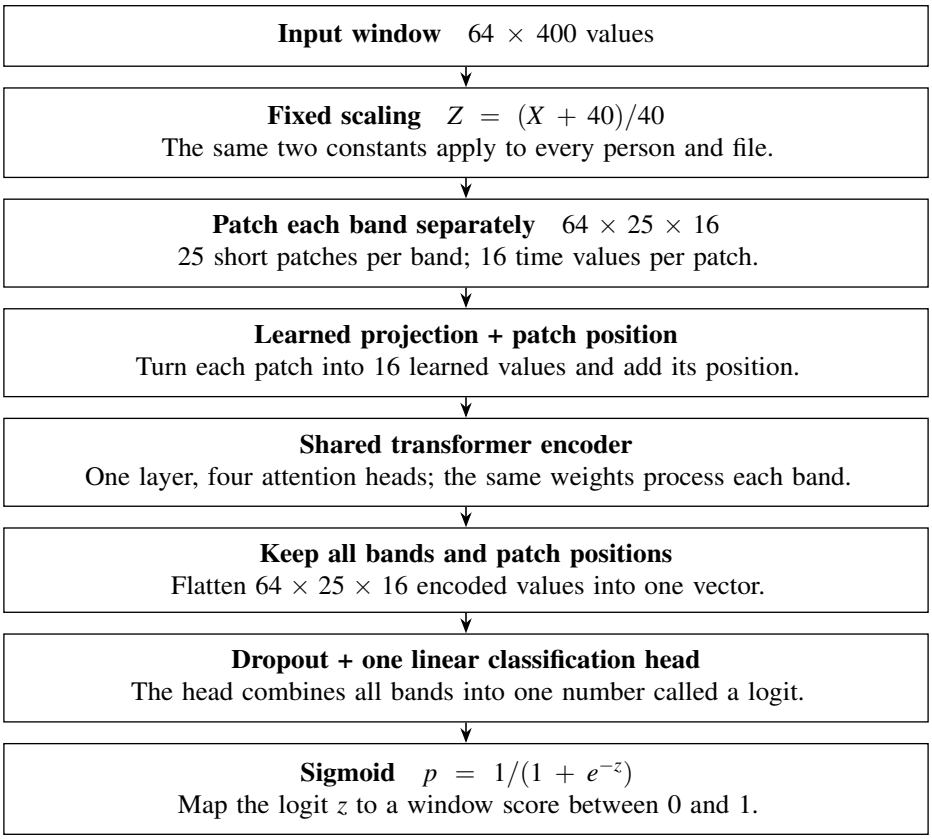


Figure 7: The single classifier used in this report. Dimensions exclude the batch size. The model has 29,553 trainable parameters.

Attention lets a patch use information from other patches in the same band. Four heads provide four learned ways of comparing those patches. The encoder width is 16 and its feed-forward width is 64, with GELU activation. **Channel-independent** means that bands share the encoder's weights and do not attend across bands there. The final head combines the bands.

Position embeddings allow time order to matter; they do not prove that the trained decisions need it. The later audit found the main decisions survived strong changes to within-window temporal detail. A model score is also not a calibrated estimate of a person's clinical probability of disease.

8 OOF: how each person receives a held-out score

OOF means out-of-fold. First, the 46 people are divided into five groups. For each group, a separate model is trained using only the other four groups. That model then predicts the held-out group. The five held-out tables are joined into one table containing each person exactly once.

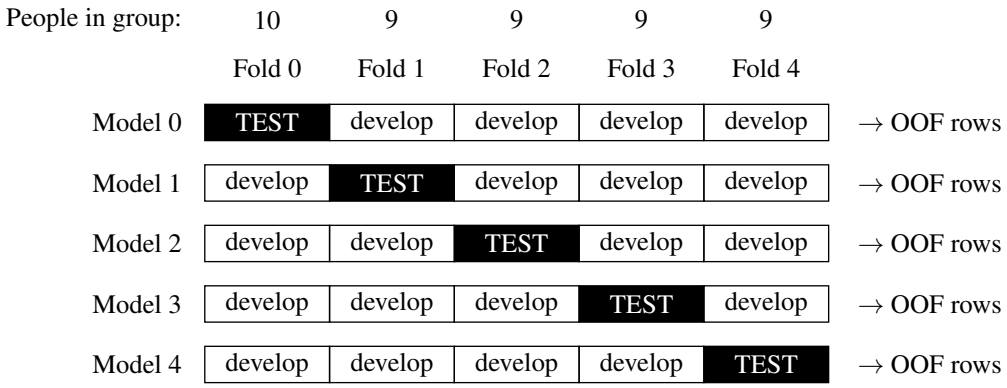


Figure 8: Each row is a separately trained model. White cells supply outer-training people, including the inner validation split. The black cell supplies that model's held-out test people. Fold IDs match the saved files.

The working policy

1. Build one row per speaker and use a stratified five-fold split with seed 42. *Stratified* means keeping both classes represented in every fold.
2. Copy the fold ID to all of that person's sessions, files and windows.
3. Hold one fold aside. Within the remaining people, choose training length using the inner procedure in Section 9.
4. Refit a fresh model on all outer-training people and predict the held-out people. The test fold does not choose weights, epochs or the threshold.
5. Save one score per held-out speaker. Repeat for fold IDs 0–4.
6. Join $10+9+9+9+9=46$ speaker rows and calculate the pooled metrics once.

What OOF does and does not mean

A person may train models for other folds. That is normal. What matters is that the model used to score that person has never trained on that person's recordings.

OOF is *not* a five-model vote for each of these 46 people. Four of those five models would have used that person during training. Averaging all five for the reported OOF score would break the held-out rule.

"Five-fold cross-validation" names the procedure. "OOF predictions" names its saved held-out outputs. They are two parts of the same evaluation, not competing methods. OOF also does not make this corpus an independent external test set.

9 Choose training length without looking at the test fold

An **epoch** is one pass through the selected training windows. Too few epochs may leave the model undertrained; too many may fit details of the training people. The project chooses the count using a small validation group taken only from the outer-training people.

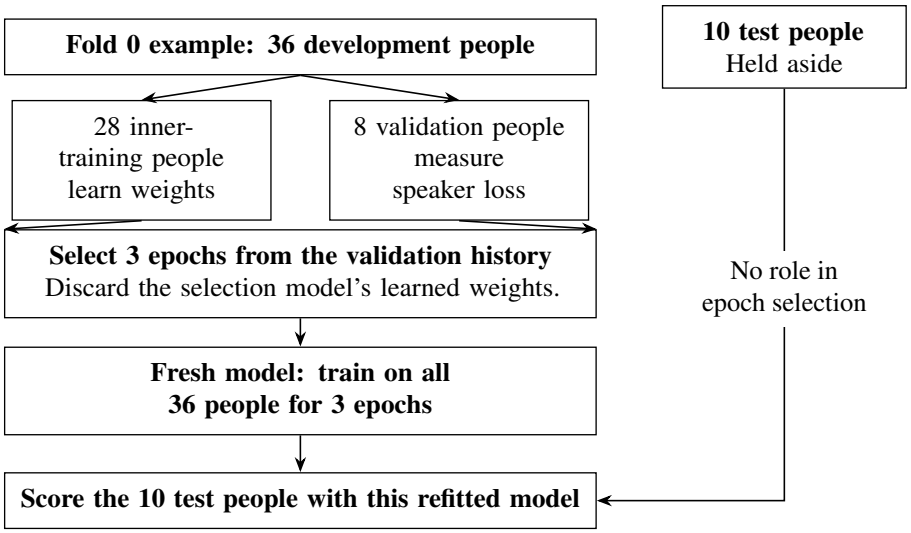


Figure 9: Actual person counts and selected epoch count for fold 0. The right-hand path joins only at final testing.

Inner validation uses a stratified 20% holdout of the outer-training people. The selection run allows at most 20 epochs. Its score is the average binary cross-entropy of *speaker* probabilities, where lower is better. An improvement must exceed 0.0001; training stops after five epochs without such an improvement. The best accepted epoch count is then used for a fresh refit.

Fold	Inner train	Validation	Refit train	Test	Chosen epochs
0	28	8	36	10	3
1	29	8	37	9	10
2	29	8	37	9	14
3	29	8	37	9	16
4	29	8	37	9	20

Validation people enter the final refit after the epoch count is fixed. Test people do not. Threshold 0.5 is fixed throughout; it is not chosen from OOF scores. The split/training seed is $42 +$ fold ID for this inner procedure.

10 How training learns from the windows

For each batch, the model produces one logit per window. Binary cross-entropy compares that output with the known label and penalizes a confident wrong answer. The optimizer changes the model's weights to reduce the weighted training loss. Labels are used as targets; they are not appended to the input spectrogram.

Why every window should not have equal influence

A long recording produces more windows than a short one. A person with two sessions produces more files than a person with one session. If all windows were treated equally, those people would have more influence just because more audio was recorded. The loss weights address this imbalance.

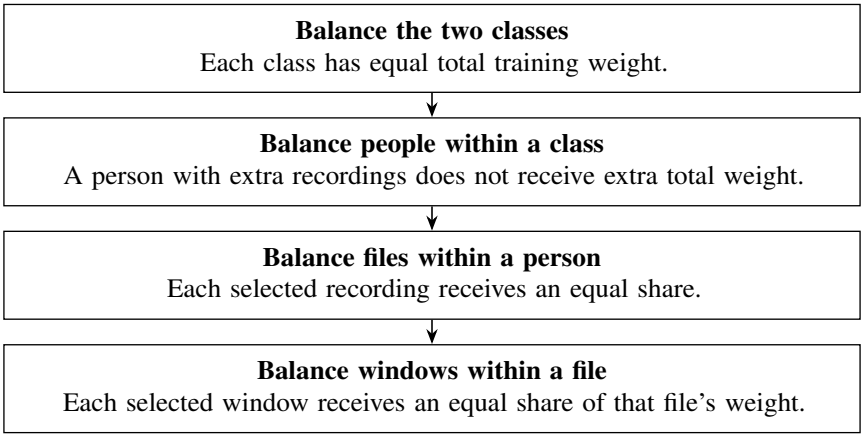


Figure 10: The weight hierarchy for training. Counts are calculated from the current training partition, not from held-out test people.

For a window belonging to person i, file j and class c, its raw weight is

$$w_{ij} = \frac{1}{N_c F_i W_{ij}},$$

where N_c is the number of training people in class c, F_i is that person's number of training files, and W_{ij} is the number of selected windows in that file. All weights are then rescaled so their average is 1. This keeps their relative importance while keeping the overall loss scale convenient.

Setting	Value	What it means
Optimizer	AdamW	Updates weights using the training loss.
Learning rate	0.001	Sets the scale of an update.
Weight decay	0.0001	Discourages unnecessarily large weights.
Batch size	16 windows	Number of windows processed in one batch.
Dropout	0.2	Randomly drops selected activations during training.
Gradient norm limit	1.0	Limits unusually large update signals.
Compute	CPU, 4 threads	The completed run used deterministic Torch settings.

During prediction the model is in evaluation mode: dropout is off, and no weights are updated. The saved checkpoint is the refitted model, not the discarded inner-selection model.

11 From window scores to one speaker decision

The model first scores short windows, but the study's question is about people. The pipeline therefore averages twice: first within a recording, then across that person's recordings. It averages probabilities, not hard yes/no votes.

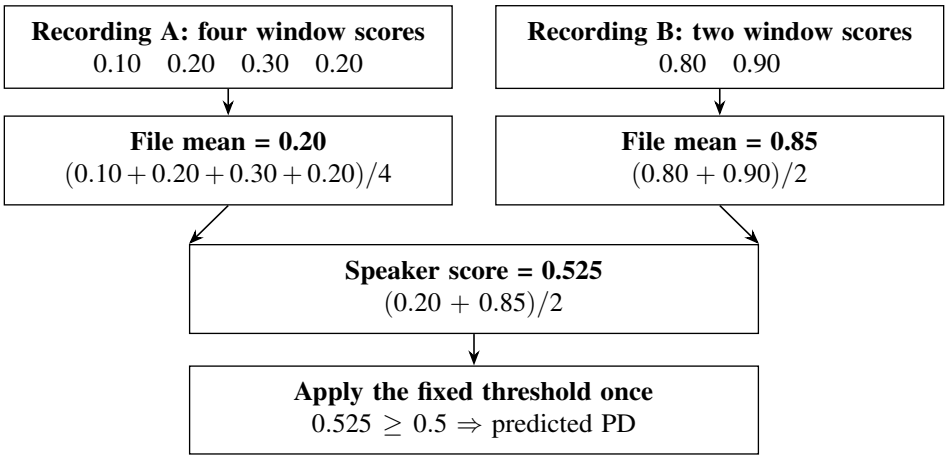


Figure 11: A made-up example to explain the arithmetic, not a real participant. Both files have equal influence on the final speaker score.

If all six windows in this example were averaged directly, the answer would be $2.50/6 \approx 0.417$. Recording A would count twice as much because it supplied twice as many windows. The two-stage average avoids that particular imbalance.

For a person with F_i files and W_{ij} windows in file j, the rule is

$$\bar{p}_{ij} = \frac{1}{W_{ij}} \sum_{k=1}^{W_{ij}} p_{ijk}, \qquad p_i = \frac{1}{F_i} \sum_{j=1}^{F_i} \bar{p}_{ij}.$$

Here p_{ijk} is a window score, $\bar{p}_{ij}$ is a file score, and p_i is the speaker score. Prediction is 1 when $p_i \geq 0.5$, otherwise 0.

Why this still needs careful interpretation

Files have equal weight, but task families do not necessarily have equal weight: a person with many vowel recordings gives that family more file slots. The main cohort also contains 10–32 recordings per person, median 16. Averaging can reduce random noise, but it can preserve a recording signature that appears across all of a person's files.

One person, one row, one decision. The final table records speaker ID, true label, probability, threshold, prediction, fold and sheet flag. Only these speaker rows enter the headline metrics.

12 Every fold, followed by the pooled result

These are the main saved results for **v2.0 / Arannamoy PatchTST / main**. All five metrics are percentages. Fold IDs are 0–4, exactly as stored in the run. Each row evaluates only the people held out from that row's model.

Fold	People	F1	Recall	Specificity	Precision	Accuracy
0	10	100.00	100.00	100.00	100.00	100.00
1	9	100.00	100.00	100.00	100.00	100.00
2	9	100.00	100.00	100.00	100.00	100.00
3	9	100.00	100.00	100.00	100.00	100.00
4	9	88.89	80.00	100.00	100.00	88.89
Mean $\pm$ SD	–	97.78 ± 4.97	96.00 ± 8.94	100.00 ± 0.00	100.00 ± 0.00	97.78 ± 4.97

Table 1: Held-out fold results and their mean $\pm$ sample standard deviation. SD describes variation across these five folds, not uncertainty in a new clinical population.

Pooled result: calculate once from all 46 held-out people

People	F1	Recall	Specificity	Precision	Accuracy
46	97.96	96.00	100.00	100.00	97.83

	Predicted PD	Predicted control
Actual PD	24 detected (TP)	1 missed (FN)
Actual control	0 false alarms (FP)	21 correctly left out (TN)

Figure 12: The pooled confusion matrix: 25 patients and 21 controls.

Why pooled F1 is not the mean fold F1. Pooling first adds the confusion counts: $TP = 24$, $FN = 1$, $TN = 21$, $FP = 0$. Then

$$\text{F1} = \frac{2TP}{2TP + FP + FN} = \frac{48}{49} = 97.96\%, \quad \text{Accuracy} = \frac{24 + 21}{46} = 97.83\%.$$

The mean fold F1 is 97.78%, because it averages five percentages instead. Both are useful; the pooled five-metric result is the primary result.

The one error. Fold 4 misses one patient at speaker probability 0.4159119, below the fixed 0.5 threshold. That fold detects four of its five patients and correctly classifies all four controls. No threshold is changed after seeing this error.

13 Reading the speaker scores and the five metrics

Figure 13 plots the actual saved OOF probabilities. No model is trained to draw it. The horizontal dashed line is the same 0.5 threshold used for every fold.

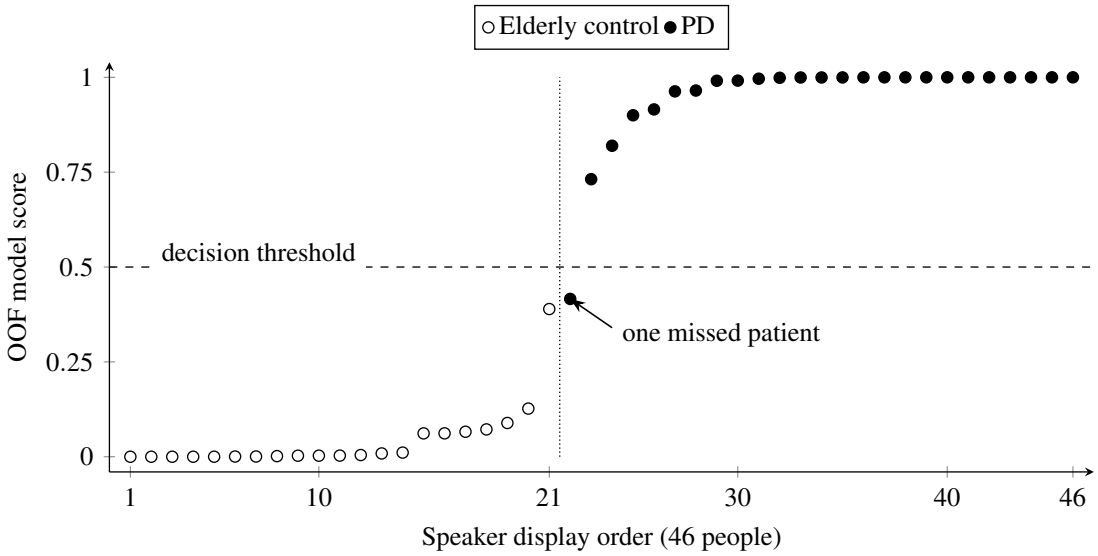


Figure 13: Actual main-cohort scores. Open circles are controls; filled circles are patients. For display only, people are grouped by true class and ordered by score within each group. This ordering is not a training input. One filled circle falls below the decision threshold.

Metric	Formula	Question it answers
F1	$2TP/(2TP+FP+FN)$	How well are precision and recall balanced?
Recall	$TP/(TP+FN)$	Of the patients, how many were detected?
Specificity	$TN/(TN+FP)$	Of the controls, how many were correctly classified?
Precision	$TP/(TP+FP)$	Of the people predicted as PD, how many were PD here?
Accuracy	$(TP+TN)/N$	Of all people, how many were classified correctly?

F1 leaves out true negatives, so it cannot replace specificity. Accuracy can hide missed patients, so it cannot replace recall. Reporting the five together shows the different types of success and error.

Precision depends on how common PD is in the evaluated group. This sample is close to half patients; its precision is not the positive predictive value of screening a general community. The probability axis likewise shows model output, not proven clinical risk.

The sample is small. One extra missed patient changes recall by four percentage points; one extra false alarm changes specificity by about 4.76 points. The five fold models also share much of their training data. Fold variation should not be read as five independent clinical trials.

14 Data leakage: the routes and the controls

Data leakage means that information from the held-out evaluation enters learning or model selection in a way it should not. A high score can then reflect prior exposure rather than a fair prediction on an unseen person.

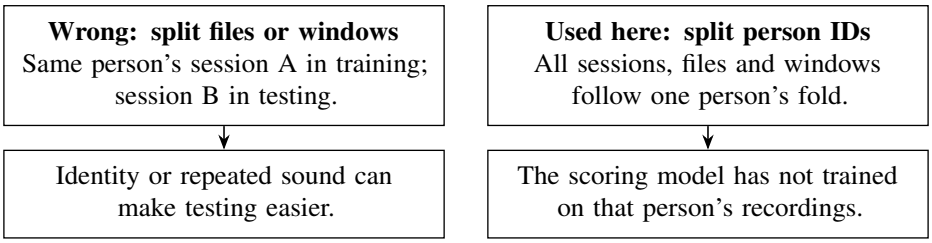


Figure 14: Why the split must follow people. Speaker separation addresses identity overlap; it does not automatically address all recording differences.

Possible route	What the pipeline does about it
Same person in train and test	Link known repeat sessions before splitting; check disjoint speaker IDs in outer and inner splits.
Overlapping audio windows	Create windows within already assigned speaker groups; never randomly divide windows across train and test.
Identical audio under another ID	Compare raw-file and decoded-audio hashes; no identical decoded signal was found across the 773 main files under different people.
Test set chooses training length	Choose epochs on inner validation speakers; refit before the held-out fold is scored.
Test set chooses the threshold	Fix the threshold at 0.5 for all folds.
Preprocessing learns from test people	Use fixed per-recording audio transforms and fixed model scaling; there is no cohort-fitted normalizer in this PatchTST.
Metadata reveals the answer	Keep IDs, labels, dates, age, sex and rate out of the feature tensor. Labels enter the training loss only.
Wrong or stale feature cache	Cache names use the audio bytes' hash and feature settings. Check shape and finite values; the audit recomputed all main features.
Repeated tuning on reported results	Record earlier development and freeze the current protocol. This history cannot be undone by rerunning the same folds.

Why caching every file is allowed here. A fixed transform can process a held-out file without learning from it, just as a saved model must eventually read a test input. If future preprocessing learns a mean, feature selection or any other parameter from multiple people, that learning must occur inside the training partition and then be applied unchanged to its held-out partition.

Remaining distinction. Recording-condition confounding can survive every identity check. Different people can still share class-related microphone or room cues. Section 15 explains the evidence for that concern.

15 What the extra checks tell us about this same model

The project keeps one primary model and the main pooled result. Two checks of the same model ask whether conclusions depend on flagged controls or on files originally recorded at another sample rate.

Evaluation	People	F1	Recall	Specificity	Precision	Accuracy
Main evaluation	46	97.96	96.00	100.00	100.00	97.83
Flagged controls excluded	43	97.96	96.00	100.00	100.00	97.67
Native 16 kHz retraining	39	100.00	100.00	100.00	100.00	100.00

Table 2: Three views of the same PatchTST procedure, all at speaker level. These are checks on the primary result, not three competing models.

Flag exclusion removes the three flagged controls from the main OOF table. It does not retrain the model. **Native-rate retraining** removes every file originally sampled at 44.1 kHz before both training and evaluation, leaving 39 people and 613 files. It retains the existing person-fold assignments.

The sole patient missed in the main run is among the seven people removed by native-rate filtering. The main predictions are already perfect on the retained 39 people. The native-only retraining is real, but its perfect result is not independent confirmation or a controlled improvement on an unchanged cohort.

The remaining recording-condition problem

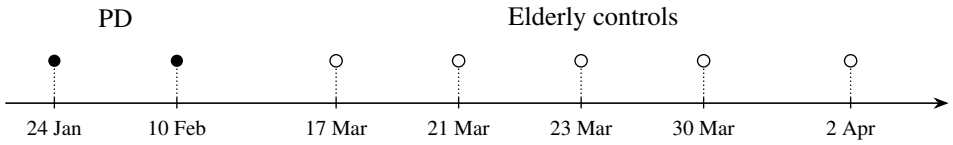


Figure 15: Native-16-kHz recording dates in 2017, ordered rather than drawn to a true time scale. Filled points are PD dates; open points are control dates. Every date has only one class.

All five native-rate speaker folds share dates between training and testing. Dates are not network inputs, but gain, microphone or room differences could be audible. Quietest-frame digital levels differ strongly between the groups; these frames have not been verified as speech-free silence.

The saved main decisions also survive shuffling frames or replacing each window with its repeated average spectrum. This means those decisions do not require the original within-window temporal detail. It does not establish that all signal is noise, or that temporal information is never useful.

How to address the remaining problem. Predeclare gain/channel/noise stress tests; verify what low-energy segments contain; and later evaluate recordings with patients and controls collected under matched conditions. Only two native PD dates exist, so five two-class, date-disjoint test folds cannot be created. External benchmarking remains the next project phase.

16 Reproducibility: connect every number to a saved file

The official trained result is v2.0. The active readable implementation is called v2.1; that code rewrite is not a new trained result. Report version 1.0 is a document version, separate from either experiment or code version.

The frozen result directory

`artifacts/runs/20261012T183328Z_f-all_cead14cc32f1/`

File relative to that run	What it records
`config.yaml`	Exact settings used by the completed run.
`protocol.json`, `folds.csv`	Cohort, source fingerprints and speaker assignments.
`source/`	Archived implementation and dependency files.
`main/arannamoy/manifest.csv`	The 773 input recordings and their metadata.
`main/arannamoy/fold_0/split.json`	Training, validation and test person IDs for fold 0.
`main/arannamoy/fold_0/selection.json`	Chosen epoch count and model size.
`main/arannamoy/fold_0/checkpoint.pt`	The refitted model's learned weights.
`main/arannamoy/speaker_predictions.csv`	The 46 pooled OOF speaker rows.
`main/arannamoy/metrics.json`	Pooled metrics, fold metrics and their spread.

The same per-fold files exist for fold IDs 1–4. Each training run stores an inner-selection history and a refit history. File hashes are fingerprints of file contents; they help detect accidental changes but do not establish that a scientific interpretation is correct.

What the latest audit actually reproduced

Audit v2.0-audit.2 recomputed all 773 main log-mel arrays from raw audio. With one BLAS arithmetic thread, all arrays matched the cache bit for bit. All ten PatchTST checkpoints, five main and five native-only, reproduced their saved decisions using freshly extracted inputs. A tiny initial discrepancy of about 1.5×10^{-5} dB was traced to arithmetic thread count. The initial failure and its resolution are both retained.

This is a raw-input and prediction replay check, not a full new training run. It confirms that the saved model, inputs and reported outputs agree. It does not remove recording-condition confounding.

How to make a new run

From the project root, prepare a fresh run, then replace `NEW_RUN` below with the directory printed by the preparation command:

```
uv sync --locked
uv run python scripts/compare_members.py --prepare
uv run python scripts/compare_members.py --run NEW_RUN --member arannamoy
```

The last command runs this member's main and native-rate scenarios. A v2.1 run gets its own results; it is not the completed v2.0 experiment. Historical checkpoints load through `PatchTST.load_v2_checkpoint`.

17 Key terms and the final interpretation

Term	Meaning in this project
Speaker	One person, even if that person has several sessions or folders.
Recording	One WAV file from one speech task.
Native sample rate	Samples per second in the original WAV, before conversion.
Frame	One short frequency-energy measurement, taken every 10 ms.
Mel band	A frequency range used to summarize sound energy.
Log-mel	Mel-band power expressed on a fixed logarithmic scale.
Window	The 400-frame input given to the model at one time.
Patch	A 16-frame piece of one band inside that window.
Parameter	A number the model learns during training.
Epoch	One pass through the selected training windows.
Validation	Training-side people used to choose how long to train.
Test fold	People excluded from the model used to predict them.
OOF prediction	A score made by the model trained without that person's fold.
Pooled result	Metrics recalculated after joining all held-out speaker rows.
Leakage	Held-out information improperly entering fitting or selection.
Confounding	Another group-related difference can help explain the score.
Checkpoint	A saved copy of the model's learned weights.

What is established

The pipeline consistently converts audio into the stated input, keeps known repeat sessions together, chooses epochs inside training people, and reports one held-out decision per speaker. The main saved result is 24 detected patients, one missed patient, 21 correctly classified controls and no false alarms. The published tables in this report are generated from saved speaker scores.

What is not established

The small, repeatedly studied corpus does not establish performance in a new clinic. Keeping people separate does not make recording conditions identical. The high score does not show that temporal attention is necessary, and the perfect native-rate check is not an independent replication. These limits are part of the result's meaning, not a reason to hide the result.

The main lesson

A trustworthy speech-classification report must explain *who was held out*, *what the model could see*, and *how the final person-level decision was made*. A high percentage alone does not answer those questions.

References

[1] G. Dimauro and F. Girardi. *Italian Parkinson's Voice and Speech*. IEEE DataPort, 2019. DOI: 10.21227/aw6b-tg17.

[2] G. Dimauro et al. "VoxTester, software for digital evaluation of speech changes in Parkinson disease." *IEEE International Symposium on Medical Measurements and Applications (MeMeA)*, 2016. DOI: 10.1109/MeMeA.2016.7533761.

[3] G. Dimauro, V. Di Nicola, V. Bevilacqua, D. Caivano and F. Girardi. "Assessment of Speech Intelligibility in Parkinson's Disease Using a Speech-To-Text System." *IEEE Access*, vol. 5, pp. 22199–22208, 2017. DOI: 10.1109/ACCESS.2017.2762475.

[4] Y. Nie, N. H. Nguyen, P. Sinthong and J. Kalagnanam. "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers." *ICLR*, 2023. arXiv:2211.14730.

[5] M. Hireš, P. Drotár, N. D. Pah, Q. C. Ngo and D. K. Kumar. "On the inter-dataset generalization of machine learning approaches to Parkinson's disease detection from voice." *International Journal of Medical Informatics*, vol. 179, 105237, 2023. DOI: 10.1016/j.ijmedinf.2023.105237.

[6] A. Mondal. Project evidence, experiment v2.0 and audits v2.0-audit.1–2, 2026. The frozen data and code provenance is identified in Section 16. Supporting project documents:
`docs/corrected_comparison.md`
`docs/corrected_results.md`
`docs/high_score_audit.md`
`docs/publication_readiness_audit.md`

Evidence and build notes

The exact model settings come from the frozen run's configuration, not from a later edit to the active configuration. Fold metrics, selected epochs and speaker probabilities are read from saved artifacts. No training takes place when this report is built.

The complete audit record for the raw-audio recheck is under:
`artifacts/runs/20261012T215648Z_v2.0-audit.2_2167119cc167/`

The report source and build instructions are under:
`docs/whitepaper_v1_0/`

Document and figures

This report uses a restrained, single-column whitepaper layout with serif text, black rules, equations and diagrams. The speaker-score plot uses actual saved predictions. The worked averaging example and input-shape sketches are illustrative. The PDF, source and build manifest are supplied together so the report can be checked and updated.

This report describes an internal, revised evaluation after prior development on the same corpus. Broader cross-dataset concerns are discussed in Hireš et al. [5]. The next benchmark phase should use a frozen procedure and clearly state which data were available during development.